

Basic Intensity Transformation Functions

What is an Intensity Transformation?

- An **intensity transformation** changes the **brightness (intensity)** values of pixels in an image.
 - It does **not** move pixels
 - It does **not** use neighboring pixels
 - It only changes **how bright or dark each pixel is**
- That's why these are also called **point operations**.

Why Do We Need Intensity Transformations?

Real images may suffer from:

- Poor lighting
- Low contrast
- Over-brightness
- Hidden details

Intensity transformations help to:

- Improve visibility
- Highlight important regions
- Prepare images for further processing

The **basic and most important ones** are:

- **Image Negative (Inversion)**
- **Power-Law (Gamma) Transformation**
- **Contrast Stretching**
- **Thresholding**
- **Histogram Equalization**

Linear Filtering and Its Applications

What is Filtering?

- An **image** is just a grid of numbers.

52	55	61
59	79	80
62	75	78

- Each number = **brightness of a pixel**.
- Modifying these numbers in a systematic way to improve the image or extract information.

Why Do We Need Filtering?

Real images often have problems:

- Noise (random bright/dark dots)
- Blur
- Poor contrast
- Hidden edges
- Unclear structures

Filtering helps to:

- Smooth noise
- Sharpen details
- Detect edges
- Prepare images for computer vision tasks

What is Linear Filtering?

Linear filtering computes each output pixel as a weighted sum of its neighboring pixels.

- Uses neighbors
- Uses weights
- Uses addition and multiplication only (that's why it's called *linear*)

Neighborhood Concept:

Look at one pixel in an image:

a	b	c
d	e	f
g	h	i

- **e** is the **current pixel**
- **a,b,c,d,f,g,h,i** are its **neighbors**

Linear filtering uses **all these values together**.

Kernel / Mask / Filter Matrix

- To decide **how much each neighbor contributes**, we use a **kernel** (also called mask or filter).
- Example kernel (3×3 mean filter):

$$\begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

- This kernel:
- Treats all neighbors equally
- Performs averaging

How Linear Filtering Works

- **Step 1: Place the kernel on the image**
 - Center the kernel on the pixel you want to compute.
- **Step 2: Multiply**
 - Multiply each image pixel by the corresponding kernel value.
- **Step 3: Add**
 - Add all the results.
- **Step 4: Replace**
 - The sum becomes the **new pixel value**.

- Mathematical Form

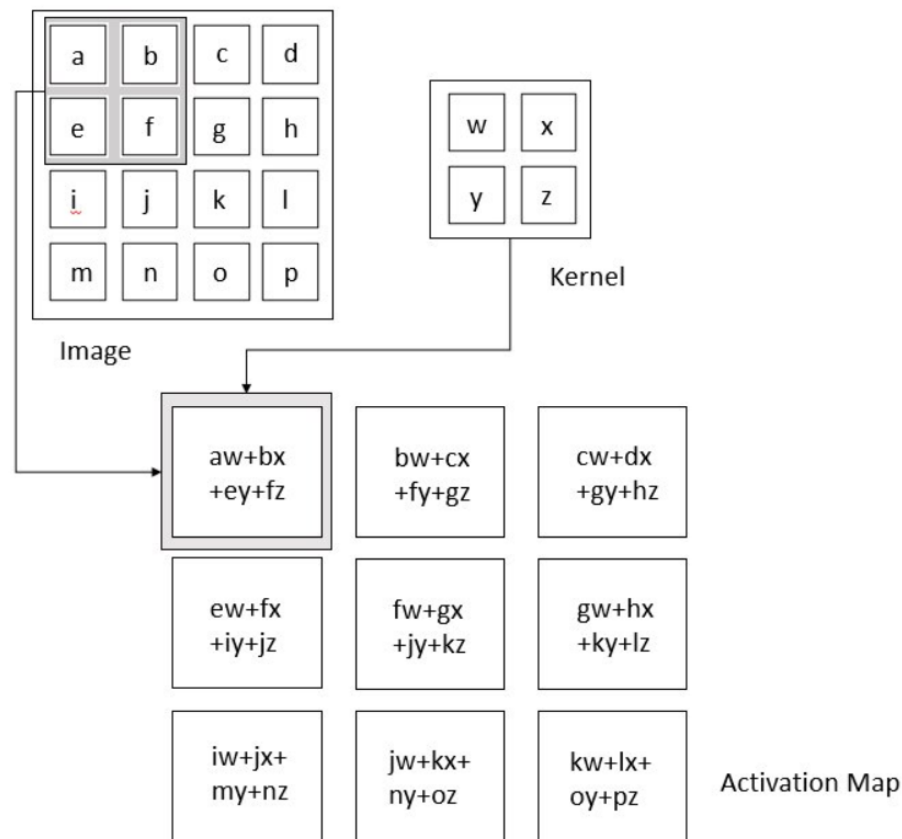
For a pixel at position (x, y) :

$$g(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k h(i, j) f(x + i, y + j)$$

Where:

- f = input image
- g = output image
- h = kernel (filter)
- This operation is called **convolution**

Every image is considered as a matrix of pixel values.



1	0	1
0	1	0
1	0	1

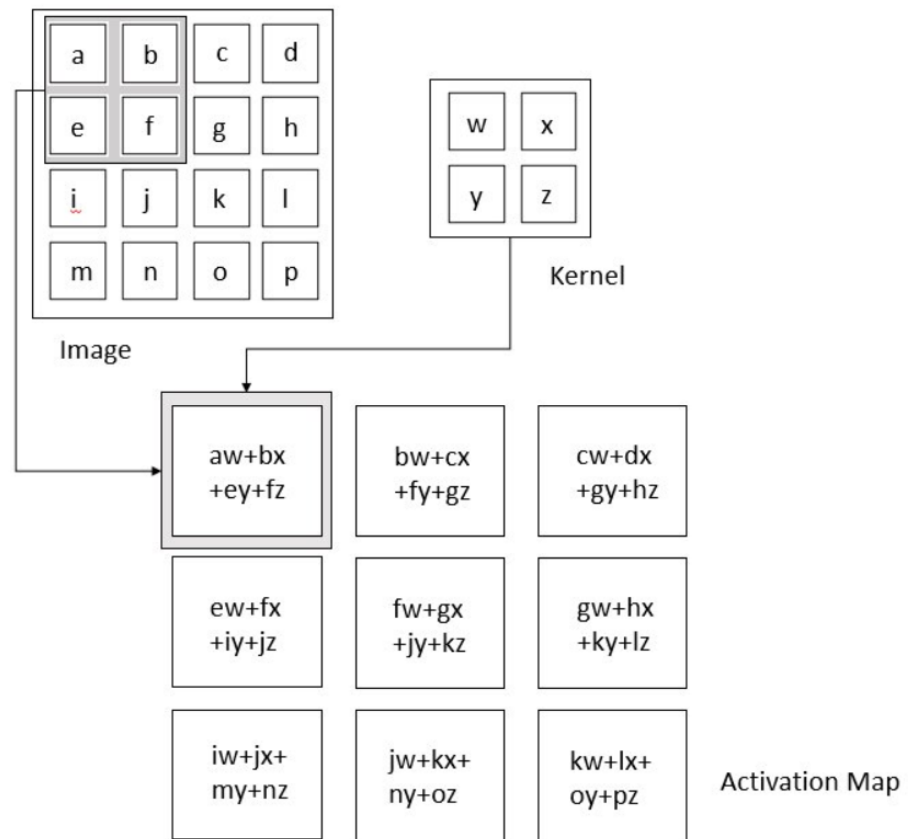
3 x 3
Filter or Kernel

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature



1	0	1
0	1	0
1	0	1

3 x 3
Filter or Kernel

1	1 _{x1}	1 _{x0}	0 _{x1}	0
0	1 _{x0}	1 _{x1}	1 _{x0}	0
0	0 _{x1}	1 _{x0}	1 _{x1}	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	

Convolved
Feature

Filters

All these filters are formed based on **one simple idea**:

- What should the center pixel become after looking at its neighbors?

Each filter answers this question **differently**, based on its goal:

- Smooth?
- Sharpen?
- Detect edges?

Types of Linear Filters

Mean Filter

$$\begin{matrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{matrix}$$

- **Effect:**

- Replaces pixel by **average of neighbors**
- Smooths noise
- Blurs edges

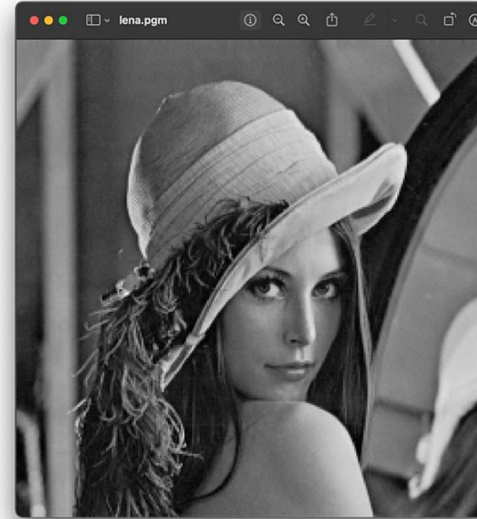
- **Use:**

- When noise is mild
- When speed is important

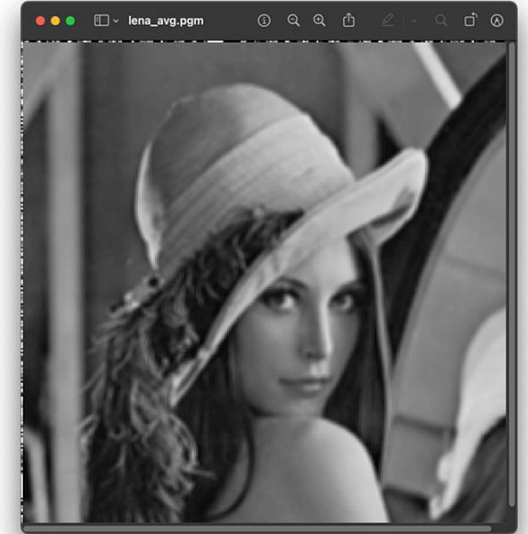
Results (including pictures):

Process results of "lena.pgm":

Source Image:



Result after average filter:



What you see

- Random noise reduced
- Image becomes slightly blurry
- Edges are softened

Why this happens

- All neighboring pixels contribute equally
- High-frequency noise gets averaged out

Mean Filter – *Formed from Averaging*

What problem does it solve?

- Noise (random bright/dark pixels)
- Small intensity fluctuations

Core Idea: Replace the pixel with the **average of itself and its neighbors**.

For a 3×3 neighborhood:

$$\text{Average} = \frac{\text{sum of all pixels}}{9}$$

This directly gives the filter.

Since there are 9 pixels:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Understanding of a Mean filter:

- All weights are equal → **no preference**
- Neighbors treated the same as center
- High-frequency variations (noise) are reduced
- **If all numbers are same → smoothing filter**

Intuition: Like asking **9 people** and taking the **average opinion**.

Gaussian Filter

- Mean filter blurs edges too much
- All neighbors are treated equally in Mean filter (not realistic)

Closer pixels should influence the center pixel **more** than distant pixels.

- This comes from **Gaussian (bell-shaped) distribution**.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- Center has maximum weight
- Weights decrease smoothly outward

Gaussian Filter

Kernel example:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Why Gaussian?

- Gives more weight to center
- Less blurring of edges
- More natural smoothing

original



filtered (Gaussian filter)



Changes

- Noise reduction
- Less edge blurring than mean filter
- Natural smooth appearance

Reason

- Center pixel has higher weight
- Far pixels contribute less

How to understand Gaussian filter

- Center weight is highest (4)
- Neighbors decrease symmetrically
- Sum = 1 $\rightarrow$ brightness preserved

Smooth + edge-preserving

- **Intuition:** Like listening **more carefully to closer people**, less to far ones.

Sharpening Filter

What is sharpening?

- Make edges and details stand out more

Edges are captured by **second derivatives** → Laplacian

Laplacian filter:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

This detects changes, but:

- Result looks like edges only
- Original image details lost

Sharpening =

Original + Edges

So add identity matrix:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

understanding Sharpening filter

- Center weight is **large positive**
- Neighbors are **negative**
- Flat regions cancel out
- Edges get amplified

Large center + negative neighbors = sharpening

Sharpening Filters:

Purpose:

- Enhance edges
- Highlight fine details

Laplacian-based Sharpening

Kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Effect:

Emphasizes intensity changes
Makes edges stand out

Warning: Also amplifies noise.



What is the reason:

- Edges become brighter
- Fine details pop out
- Image looks crisper

Why this:

- Center pixel amplified
- Neighbors subtracted
- Flat areas cancel, edges remain

Edge Detection Filters

Edges = places where brightness changes rapidly.

Sobel Filter (Gradient-based)

Sobel-X:

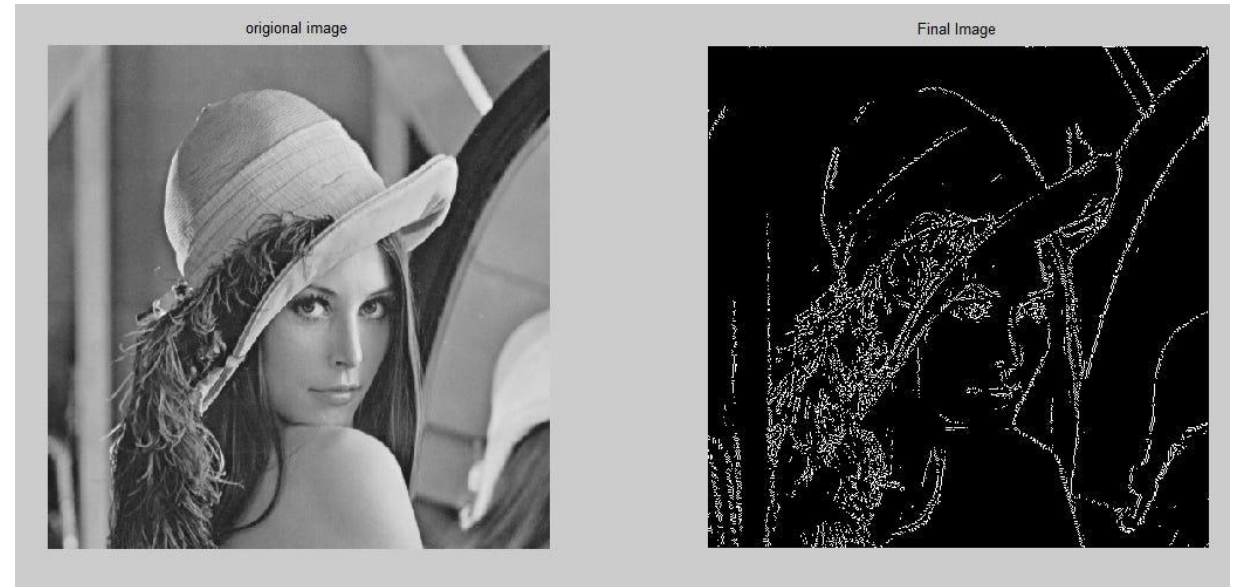
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel-Y:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Effect:

Detects vertical and horizontal edges
Foundation for object detection



What you see

- Object boundaries highlighted
- Flat regions disappear
- Only transitions remain

Why this happens

- Kernel sums to zero
- Detects intensity changes (derivatives)

Understanding edge filters

- Opposite signs $\rightarrow$ difference
- Zero-sum kernel $\rightarrow$ flat areas vanish
- Strong response only at edges

If kernel sums to zero $\rightarrow$ edge detector

Applications of Linear Filtering

- **Noise Removal**

- Mean / Gaussian filter
- Medical images
- Satellite images

- **Edge Detection**

- Sobel, Prewitt, Laplacian
- Object boundaries
- Lane detection

- **Image Sharpening**

- Photography
- Remote sensing
- Surveillance

- **Feature Extraction**

- Corners
- Textures
- Shapes (preprocessing for ML/CV)

- **Computer Vision Pipelines**

- Segmentation
- Object detection
- Recognition
- Deep learning